Capstone Scope — KijaniKiosk kk-payments

Track A: Infrastructure-First · Victor Kanyingi · Version 1.1 · 10 Nov 2026

1. The problem I am solving

Up to Week 8, every time I deployed kk-payments it went **straight into the only namespace I had (`default`)**, which acts as production. I never had a place to try a new version first, and nothing recorded who approved a release or why. I can show this today. `kubectl get ns` has no staging namespace. My Week 5 Jenkins pipeline stops after publishing to Nexus and never deploys. My Week 8 deployment has no approval record. And if a kk-payments pod keeps crashing, nothing tells me unless I happen to be watching.

2. What I will build (in scope)

1. A `kijani-staging` namespace with resource quotas and limits, created by Terraform so I can rebuild it from scratch.
2. Ansible playbooks that give each environment its own settings (for example a different `DB_HOST`), so the **same** Deployment file works in both.
3. A Jenkins pipeline that deploys every merge to `main` to staging, tests it there, and only then asks a person to approve production, recording who approved and why.
4. Prometheus alert rules that fire when kk-payments pods keep restarting or when payments start failing.
5. Staging kk-payments saving each receipt to `kk-payments-receipts-staging`, which starts my Week 10 receipt chain (kk-processor → kk-notifier).

3. What I am leaving out, and why

Left out	Why
Separate clusters for staging and production	I only have one laptop running one Minikube node. Namespaces with resource limits are the closest I can get, and I list the remaining risk in the README.
Deploying to a real cloud provider	I have done the whole course without a cloud account. Keeping everything local means anyone can run my project for free.
Grafana dashboards	Alerts tell me when to act, and dashboards only show me information. My laptop's memory is already tight with Minikube, Jenkins and Prometheus running together.
A real database	A different `DB_HOST` per environment is enough to show that config is separated. Adding a database for each environment would double the work without teaching me anything new for this project.

4. How I will know it works (each one shown in the demo)

#	Success criterion	What I will show
S1	I can rebuild the infrastructure from a clean copy of the repo	`terraform apply` creates the namespace, quotas and limit ranges, and running `terraform plan` again says **No changes**
S2	Merging to `main` starts the pipeline with no button press	After I merge a PR, Jenkins starts a build within **3 minutes**, and staging `/health` shows the new `<version>-<commit>`
S3	Production approval only appears after staging passes, and it records a reason	*Approve Production* appears only after *Smoke Test Staging* is green. `approval-record.json` and the production Deployment show who approved it and a reason of at least 10 characters
S4	One Deployment file, two sets of settings	`/health` shows `db_host: kk-db.kijani-staging…` in staging and `kk-db.default…` in production, and `k8s/` holds only one Deployment file
S5	A bad change is stopped safely	I merge a branch with a bug. The staging test fails, staging is rolled back, approval is never offered, and production keeps its old version. A crash fault fires the `KkPaymentsPodRestarting` alert within **5 minutes**
S6	The receipt chain fires from Kubernetes	One staging payment creates `receipt-<orderId>.json`, and within **30 seconds** kk-processor and kk-notifier both log the same `orderId`

5. Architecture

See `docs/architecture.png`. Every arrow is labelled with what passes along it. *v1.1: after re-reading my Week 10 labs, the receipt buckets moved from Terraform/MinIO into the serverless stack (ADR-004).*